

KMS DATASET KNOWLEDGE GRAPH

Nilzelly Caroline Martins da Silva

INTRODUCTION

- This presentation shows how I turned the GitHub CSV dataset into a small knowledge graph, checked the import and used Cypher queries to understand the study programmes.
- The goal was not only to import CSV files, but to make the relationships visible and queryable.

OBJECTIVES

- Explore the dataset from the GitHub repository.
- Build a graph schema from the node and edge CSV files.
- Import the graph into Neo4j AuraDB Free.
- Run Cypher queries and explain what the results mean.

WHY IS A KNOWLEDGE GRAPH IMPORTANT?

The dataset is about connections: programmes have courses, courses have topics and courses also have learning outcomes. Neo4j lets us follow those links directly instead of trying to piece them together manually from separate tables.

DATASET AND MAPING IDEA

Node files

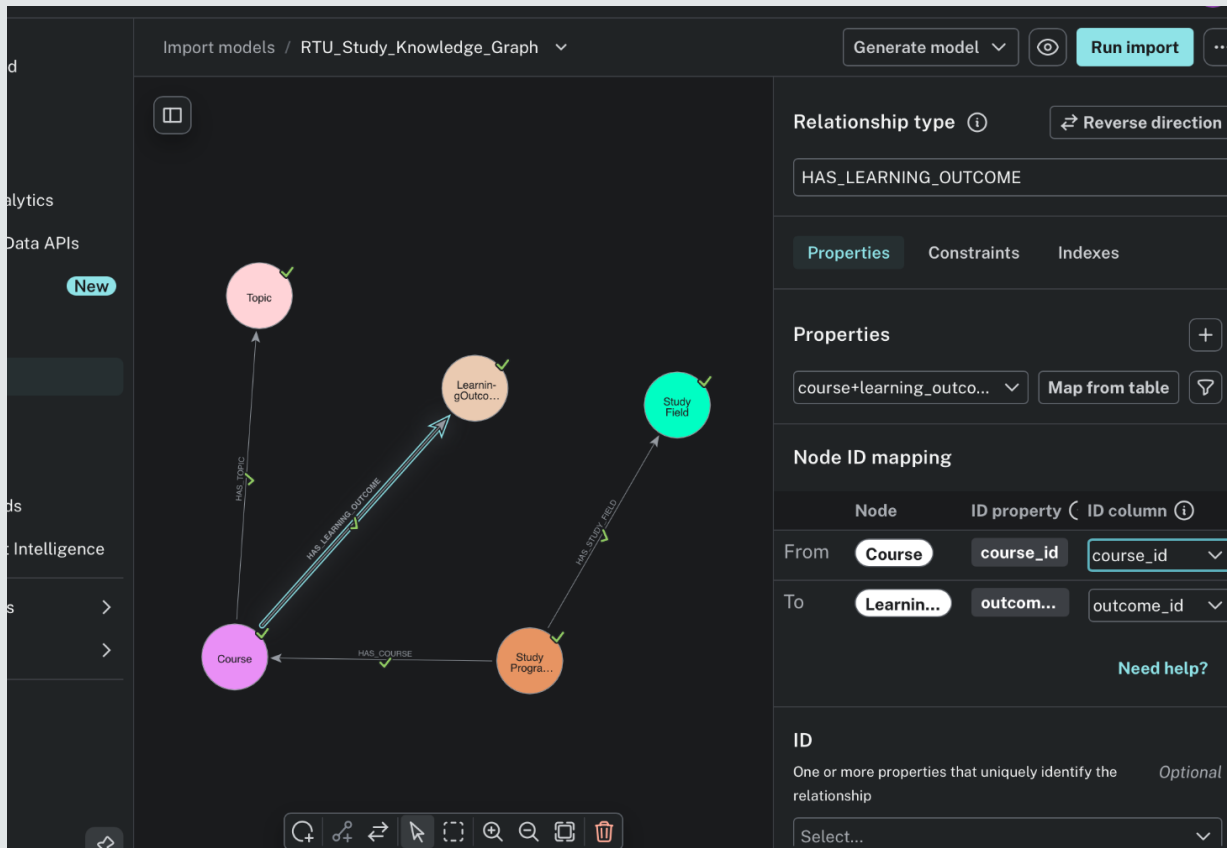
nodes_study_programs.csv	StudyProgram
nodes_study_courses.csv	Course
nodes_course_topics.csv	Topic
nodes_learning_outcomes.csv	LearningOutcome
nodes_study_fields.csv	StudyField

Relationship files

program+course.csv	HAS_COURSE
program+study_field.csv	HAS_STUDY_FIELD
course+topic.csv	HAS_TOPIC
course+learning_outcome.csv	HAS_LEARNING_OUTCOME

GRAPH SCHEMA

This is the schema I built before running the import.



How to read it

- StudyProgram connects to Course.
- Course connects to Topic.
- Course connects to LearningOutcome.
- StudyProgram connects to StudyField.

This screenshot is useful for the final submission because it proves that the schema was designed manually before importing.

IMPORT INTO AURADB

After mapping the schema, I ran the import in my AuraDB Free instance

Import results

Total time: 00:00:02

Run import completed

Import completed successfully.

Topic

nodes_course_topics.csv

Show Cypher

Time taken	File size	File rows	Nodes created	Properties set	Labels added	Query time
00:00:00	65.4 KiB	640	640	4,480	640	00:00:00

Course

nodes_study_courses.csv

Show Cypher

Time taken	File size	File rows	Nodes created	Properties set	Labels added	Query time
00:00:00	284.0 KiB	75	75	895	75	00:00:00

Close

Explore with agent

Import completed successfully

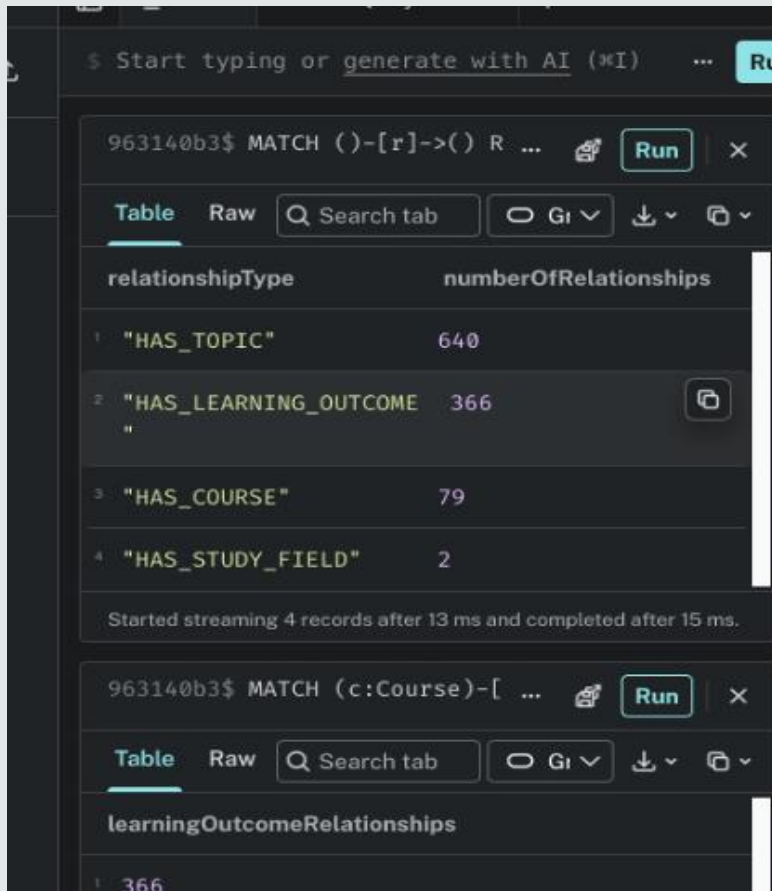
The import result confirms that the CSV files were processed and nodes were created in Neo4j. This was the point where the dataset became a real graph database.

640 Topic nodes created

75 Course nodes created

QUICK VALIDATION CHECK

Before I analyze the graph, I checked whether all relationship types were present.



The screenshot shows a Neo4j Cypher query interface. The query is `MATCH ()-[r]->() RETURN type(r)`. The results are displayed in a table with two columns: `relationshipType` and `numberOfRelationships`. The table contains four rows of data. Below the table, a status message indicates that 4 records were streamed after 13 ms and completed after 15 ms. A second query is also visible at the bottom: `MATCH (c:Course)-[r]->(o:LearningOutcome) RETURN r`, with its results showing a count of 366.

relationshipType	numberOfRelationships
"HAS_TOPIC"	640
"HAS_LEARNING_OUTCOME"	366
"HAS_COURSE"	79
"HAS_STUDY_FIELD"	2

Started streaming 4 records after 13 ms and completed after 15 ms.

learningOutcomeRelationships
366

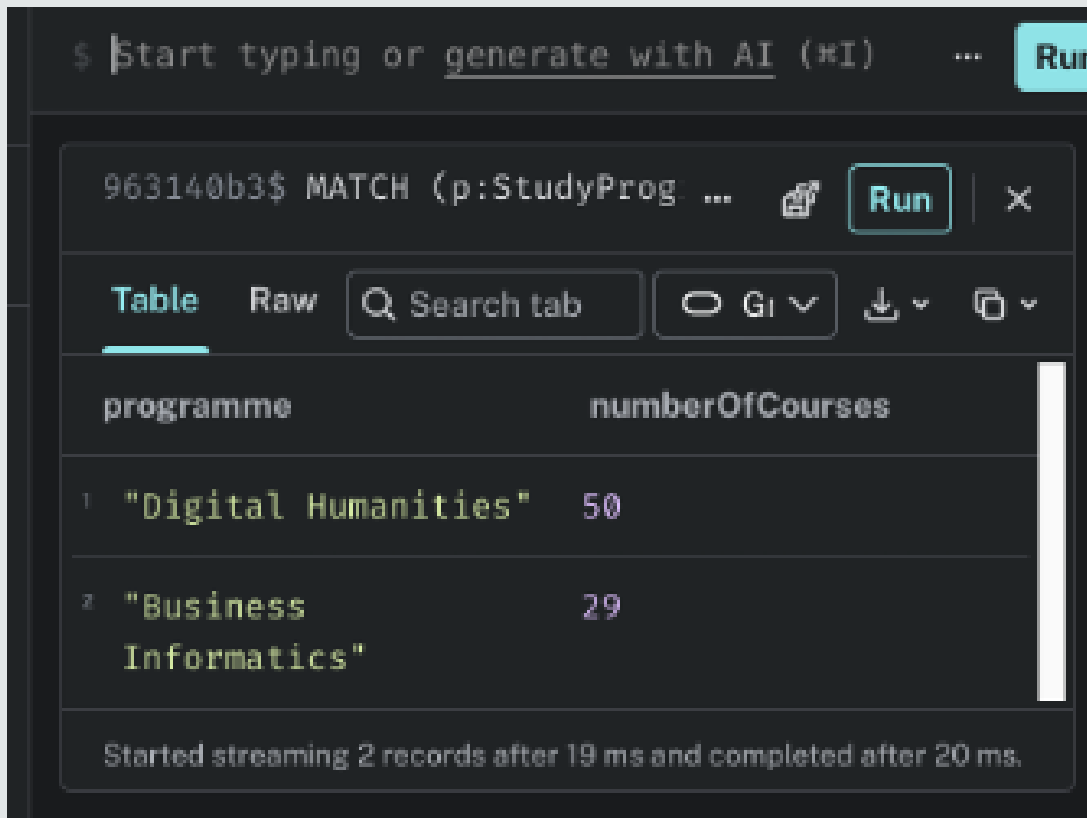
Relationship count

640	HAS_TOPIC
366	HAS_LEARNING_OUTCOME
79	HAS_COURSE
2	HAS_STUDY_FIELD

I had to correct the Course → LearningOutcome mapping once. After fixing it, the 366 relationships appeared correctly.

QUERY 1 – COURSE PER PROGRAMME

This query answers a simple question: how many courses does each programme have?



The screenshot shows a query execution interface. At the top, there is a text input field with the placeholder "\$ Start typing or generate with AI (#I)" and a "Run" button. Below this, a query editor shows the query "963140b3\$ MATCH (p:StudyProg ...". Below the query editor, there are tabs for "Table" and "Raw", a search bar, and a "Run" button. The "Table" tab is selected, and it displays a table with two columns: "programme" and "numberOfCourses". The table contains two rows: 1. "Digital Humanities" with 50 courses, and 2. "Business Informatics" with 29 courses. At the bottom, a status message reads: "Started streaming 2 records after 19 ms and completed after 20 ms."

programme	numberOfCourses
"Digital Humanities"	50
"Business Informatics"	29

Main result

50 Digital Humanities courses

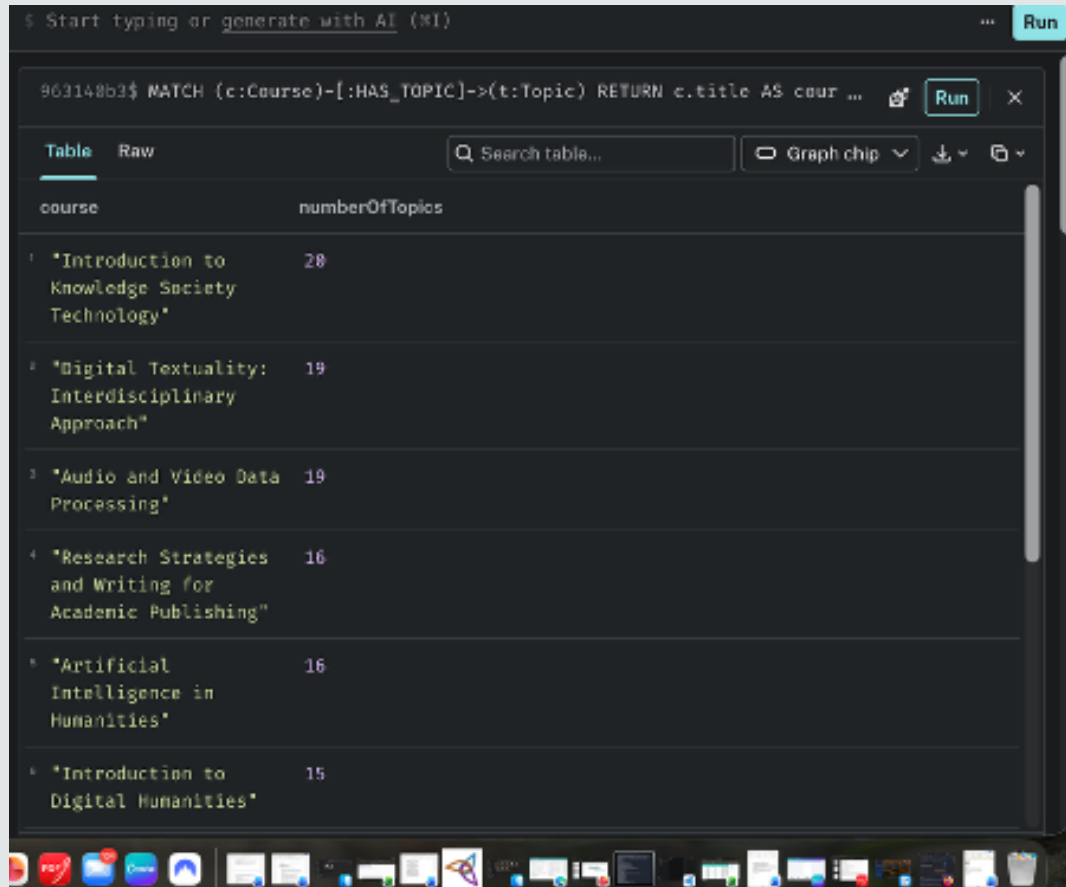
29 Business Informatics courses

In this dataset, Digital Humanities has the larger course structure.

```
MATCH (p:StudyProgram)-[:HAS_COURSE]->(c:Course)
RETURN p.title AS programme, count(c) AS numberOfCourses
ORDER BY numberOfCourses DESC;
```

QUERY 2 – COURSES WITH THE MOST TOPICS

This helped me see which courses have the most detailed topic structure.



The screenshot shows a Neo4j Cypher query interface. The query is: `MATCH (c:Course)-[:HAS_TOPIC]->(t:Topic) RETURN c.title AS cour ...`. The results are displayed in a table with two columns: 'course' and 'numberOfTopics'. The table lists six courses with their respective topic counts.

course	numberOfTopics
"Introduction to Knowledge Society Technology"	20
"Digital Textuality: Interdisciplinary Approach"	19
"Audio and Video Data Processing"	19
"Research Strategies and Writing for Academic Publishing"	16
"Artificial Intelligence in Humanities"	16
"Introduction to Digital Humanities"	15

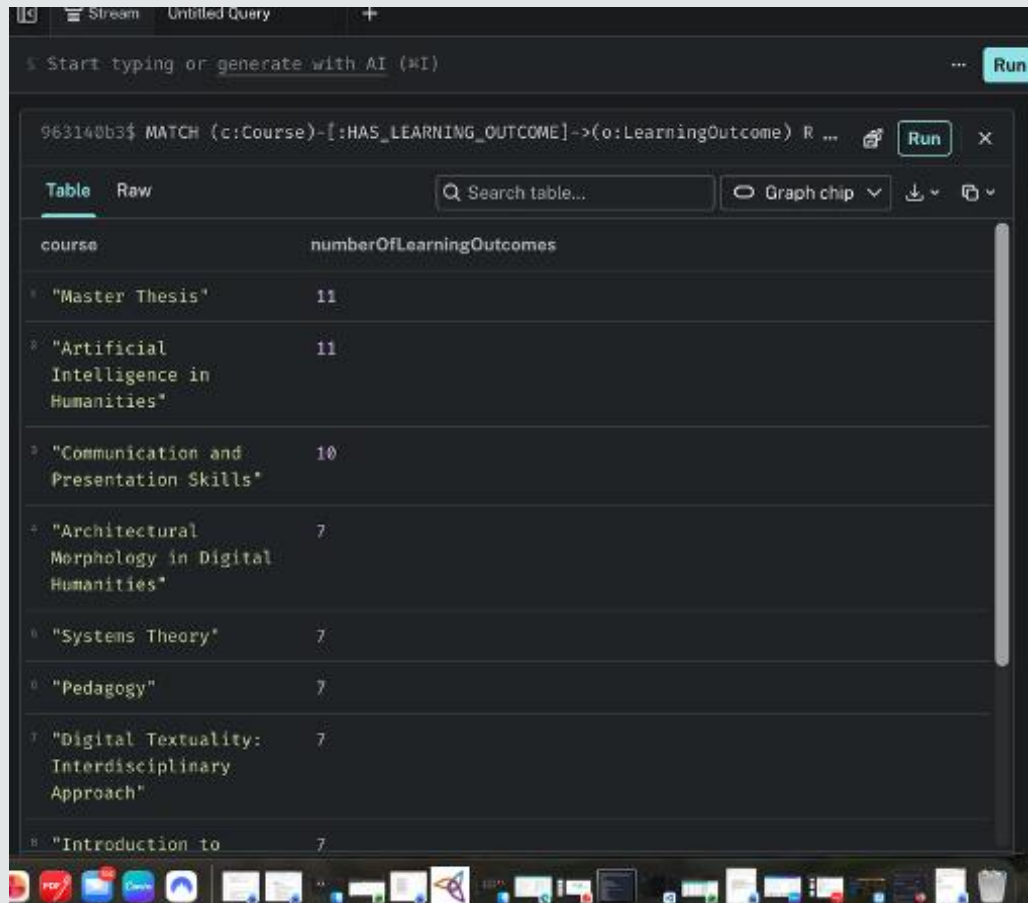
What stood out

- Introduction to Knowledge Society Technology has 20 topics.
- Digital Textuality: Interdisciplinary Approach has 19.
- Audio and Video Data Processing also has 19.

My interpretation: these courses are either broader or broken into more detailed topic units.

QUERY 3 – COURSES WITH THE MOST LEARNING OUTCOMES

This query shows where the most learning outcomes are attached.



The screenshot shows a query editor with a Cypher query: `MATCH (c:Course)-[:HAS_LEARNING_OUTCOME]->(o:LearningOutcome) R ...`. Below the query, a table displays the results. The table has two columns: 'course' and 'numberOfLearningOutcomes'. The results are sorted by the number of learning outcomes in descending order.

course	numberOfLearningOutcomes
"Master Thesis"	11
"Artificial Intelligence in Humanities"	11
"Communication and Presentation Skills"	10
"Architectural Morphology in Digital Humanities"	7
"Systems Theory"	7
"Pedagogy"	7
"Digital Textuality: Interdisciplinary Approach"	7
"Introduction to"	7

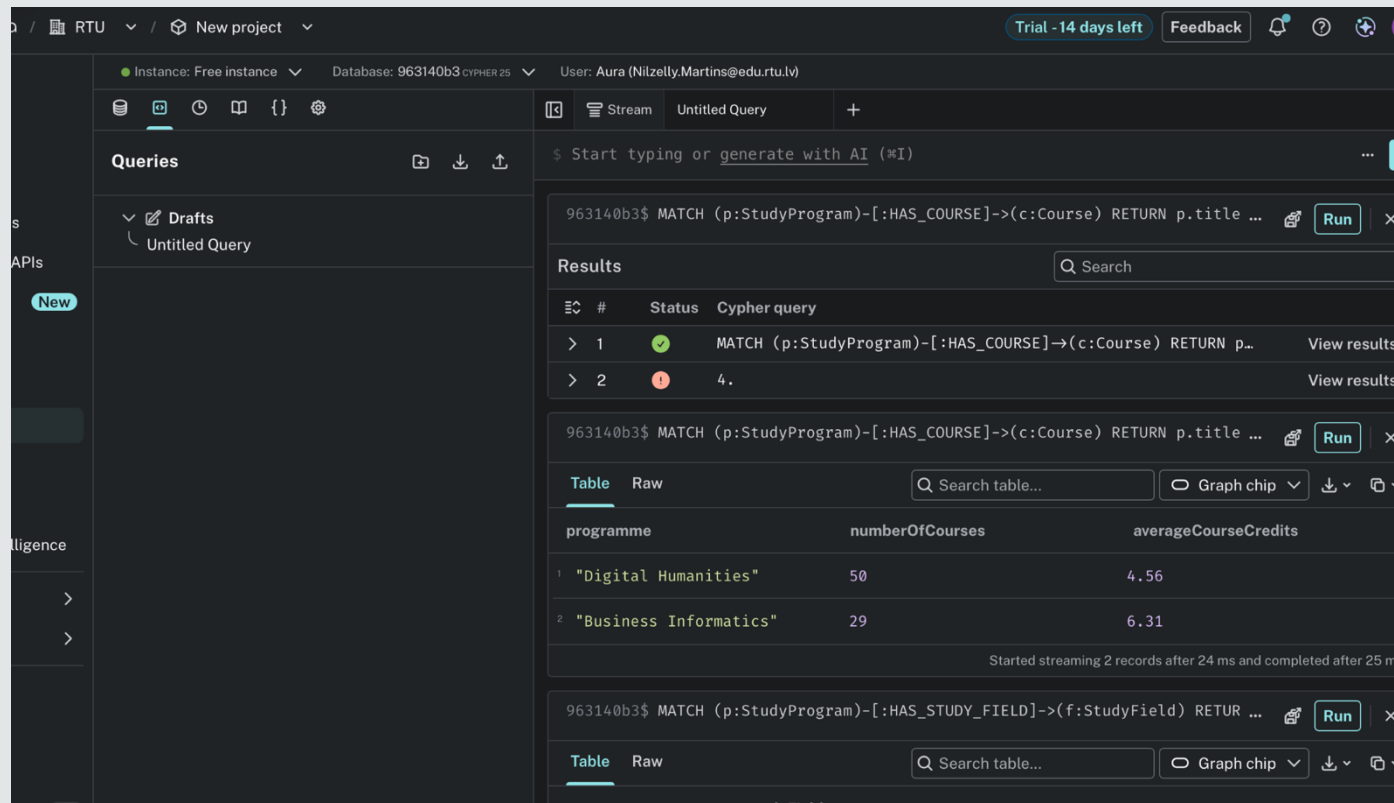
Top results

- Master Thesis has 11 learning outcomes.
- Artificial Intelligence in Humanities also has 11.
- Communication and Presentation Skills has 10.

This makes it easy to spot courses with a larger number of explicitly described objectives.

QUERY 4 – COURSE LOAD AND AVERAGE CREDITS

This query compares programme size with average course credits.



The screenshot shows a database interface with two queries and their results. The first query is a simple match query. The second query is a more complex match query that returns the number of courses and average course credits for two study programs.

```
963140b3$ MATCH (p:StudyProgram)-[:HAS_COURSE]->(c:Course) RETURN p.title ...
```

programme	numberOfCourses	averageCourseCredits
1 "Digital Humanities"	50	4.56
2 "Business Informatics"	29	6.31

A useful comparison

Digital Humanities has more courses, but the average credit value is lower.

Business Informatics has fewer courses, but the average credit value is higher.

Digital Humanities: 50 courses, 4.56 avg credits
Business Informatics: 29 courses, 6.31 avg credits

CONCLUSION

- This was my first experience working with Neo4j and knowledge graphs. At first, understanding nodes, relationships, IDs, and Cypher queries was challenging, but building the graph step by step helped me understand how connected data works.
- The final graph made the structure of the study programmes much easier to see. The analysis showed that **Digital Humanities has more courses (50) than Business Informatics (29)**, while **Business Informatics has a higher average number of credits per course (6.31 compared with 4.56)**. The graph also helped identify courses with many topics and learning outcomes.
- Overall, this project showed me how a knowledge graph can turn separate datasets into connected information and make comparisons between programmes and courses easier to understand.